Лабораторная работа №15

Тема работы: «Разработка алгоритмов и программ методом пошаговой детализации. Организация ввода- вывода с помощью текстовых файлов»

1 Цель работы

Сформировать умение разрабатывать и реализовывать программы создания, корректировки и просмотра пользовательских файлов

2 Задание

1. Создать текстовый файл. Написать программу, которая: открывает созданный файл, определяет количество слов в тексте, записывает во второй файл текст из первого в виде набора слов.

12. Создать текстовый файл. Написать программу, которая: открывает созданный файл, определяет лишние пробелы между словами, выводит во второй файл текст без лишних пробелов.

14. Создать текстовый файл. Написать программу, которая: открывает созданный файл, определяет количество слов текста, оканчивающееся на гласную букву, записывает во второй файл эти слова.

3 Оснащение ПК, задание по варианту, среда разработки Visual Studio.

4. Порядок выполнения работы.

```
#include <iostream>
#include <fstream>
#include <string>
#include <windows.h>
using namespace std;

void task1() {
    ofstream OutFile2("manyspace.txt");
    if (!OutFile2.is_open()) {
        cerr << "Ошибка открытия файла!" << endl;
        return;
    }

    string line;
    cout << "Введите данные в файл:" << endl;
    if (cin.peek() == '\n') cin.ignore();
    getline(cin, line);
    OutFile2 << line;
    OutFile2.close();

    for (size_t i = 0; i < line.length();) {
        if (line[i] == ' ' && i + 1 < line.length() && line[i + 1] == ' ') {
            line.erase(i + 1, 1);
        }
        else {
            i++;
        }
    }

    while (!line.empty() && line.back() == ' ') {
        line.pop_back();
    }

    cout << "Текст без лишних пробелов: " << line << endl;

    ofstream OutFile3("normspace.txt");
    if (!OutFile3.is_open()) {
```

```
38.         cerr << "Ошибка открытия файла!" << endl;
39.         return;
40.     }
41.     OutFile3 << line;
42.     OutFile3.close();
43.
44.     cout << "Результат записан в normspace.txt" << endl;
45. }
46.
47. void task2() {
48.     ofstream OutFile5("polny.txt");
49.     if (!OutFile5.is_open()) {
50.         cerr << "Ошибка открытия файла!" << endl;
51.         return;
52.     }
53.
54.     string line;
55.     cout << "Введите данные в файл:" << endl;
56.     if (cin.peek() == '\n') cin.ignore();
57.     getline(cin, line);
58.     OutFile5 << line;
59.     OutFile5.close();
60.
61.     ofstream OutFile6("glasnye.txt");
62.     if (!OutFile6.is_open()) {
63.         cerr << "Ошибка открытия файла!" << endl;
64.         return;
65.     }
66.
67.     int counter = 0;
68.     string word;
69.     string vowels = "аеiоuуаеёиоуыэюя";
70.
71.     for (size_t i = 0; i <= line.length(); i++) {
72.         if (i == line.length() || line[i] == ' ') {
73.             if (!word.empty()) {
74.                 char last = word.back();
75.                 if (vowels.find(last) != string::npos) {
76.                     counter++;
77.                     OutFile6 << word << endl;
78.                 }
79.             }
80.             word.clear();
81.         }
82.         else {
83.             word += line[i];
84.         }
85.     }
86.
87.     cout << "Количество слов, заканчивающихся на гласную букву: " << counter << endl;
88.     OutFile6.close();
89.     cout << "Результат записан в glasnye.txt" << endl;
90. }
91.
92. void task3() {
93.     ofstream outFile("example.txt");
94.     if (!outFile.is_open()) {
95.         cerr << "Ошибка при открытии файла!" << endl;
96.         return;
97.     }
98.
99.     string line;
100.     cout << "Введите текст для записи в example.txt:" << endl;
101.     if (cin.peek() == '\n') cin.ignore();
102.     getline(cin, line);
103.     outFile << line << endl;
104.     outFile.close();
105.
106.     ifstream inFile("example.txt");
107.     if (!inFile.is_open()) {
108.         cerr << "Ошибка открытия файла!" << endl;
109.         return;
110.     }
111.
112.     cout << "Содержимое файла example.txt:" << endl;
113.
```

```
114.     int counter = 0;
115.     string content;
116.     string current;
117.     while (getline(inFile, current)) {
118.         cout << current << endl;
119.         content += current;
120.         content += '\n';
121.         for (char c : current) {
122.             if (c == ' ') ++counter;
123.         }
124.     }
125.     inFile.close();
126.
127.     if (!content.empty() && content.back() == '\n') content.pop_back();
128.     cout << "Количество слов в файле: " << (content.empty() ? 0 : counter + 1) << endl;
129.
130.     ofstream outFile1("example1.txt");
131.     if (!outFile1.is_open()) {
132.         cerr << "Ошибка открытия файла!" << endl;
133.         return;
134.     }
135.     outFile1 << content;
136.     outFile1.close();
137.
138.     cout << "Результат записан в example1.txt" << endl;
139. }
140.
141. int main() {
142.     SetConsoleOutputCP(1251);
143.     SetConsoleCP(1251);
144.     setlocale(LC_ALL, "RU");
145.
146.     while (true) {
147.         cout << "Выберите задание:\n";
148.         cout << "1 - Удаление лишних пробелов\n";
149.         cout << "2 - Слова, заканчивающиеся на гласную\n";
150.         cout << "3 - Количество слов в тексте\n";
151.         cout << "0 - Выход\n";
152.
153.         int choice;
154.         cin >> choice;
155.
156.         if (choice == 0) break;
157.         else if (choice == 1) task1();
158.         else if (choice == 2) task2();
159.         else if (choice == 3) task3();
160.         else cout << "Неверный выбор!" << endl;
161.     }
162.
163.     return 0;
164. }
165.
```

```
Выберите задание:
1 - Удаление лишних пробелов
2 - Слова, заканчивающиеся на гласную
3 - Количество слов в тексте
0 - Выход
1
Введите данные в файл:
Привет    мир    это    тестовая    строка
Текст без лишних пробелов: Привет мир это тестовая строка
Результат записан в normspace.txt
```

Рисунок 1 - Пример выполнения задачи №12

```
Выберите задание:
1 - Удаление лишних пробелов
2 - Слова, заканчивающиеся на гласную
3 - Количество слов в тексте
0 - Выход
2
Введите данные в файл:
мама папа кот собака окно стол книга
Количество слов, заканчивающихся на гласную букву: 5
Результат записан в glasnye.txt
```

Рисунок 2 - Пример выполнения задачи №14

```
1 - Удаление лишних пробелов
2 - Слова, заканчивающиеся на гласную
3 - Количество слов в тексте
0 - Выход
3
Введите текст для записи в example.txt:
Hello world this is a simple test line
Содержимое файла example.txt:
Hello world this is a simple test line
Количество слов в файле: 8
Результат записан в example1.txt
```

Рисунок 3 - Пример выполнения задачи №1

5. Контрольные вопросы

1. Дайте краткое понятие метода пошаговой детализации при программировании.

Метод пошаговой детализации — это способ разработки программы, при котором задача разбивается на более простые подзадачи. Каждая подзадача уточняется и детализируется до тех пор, пока не станет достаточно простой для реализации в виде конкретных операторов и функций. Такой подход обеспечивает системность и упрощает процесс программирования.

2. Сформулируйте сущность организации ввода-вывода с использованием текстовых файлов в программе.

Организация ввода-вывода с использованием текстовых файлов заключается в сохранении и обработке информации в виде последовательности символов.

Программа может читать данные из файла или записывать их в файл, используя стандартные средства языка C++ для работы с потоками.

3. Изложите особенности чтения данных из текстового файла в программе на C++.

– Для чтения используется класс ifstream.

– Данные читаются построчно или посимвольно, в зависимости от задачи.

– При чтении учитываются разделители (пробелы, табуляции, символы новой строки).

– Возможна проверка состояния файла (например, конец файла с помощью eof()).

4. Раскройте особенности записи данных в текстовый файл из программы на C++.

– Для записи используется класс ofstream.

– Данные записываются в файл в виде символов и строк.

– Можно управлять режимом открытия файла: перезапись (ios::out) или добавление (ios::app).

– После завершения работы файл необходимо закрыть методом close().

5. Дайте определение потокового ввода-вывода и его взаимодействия с текстовыми файлами.

Потоковый ввод-вывод — это механизм передачи данных между программой и внешними устройствами (например, файлами) через абстракцию потока. Поток обеспечивает последовательное чтение или запись символов. Взаимодействие с текстовыми файлами осуществляется через классы ifstream, ofstream и fstream.

6. Сформулируйте особенности позиционирования в текстовом файле при чтении и записи данных на C++.

– Позиционирование осуществляется с помощью указателя текущей позиции в файле.

– Для управления используются методы seekg() (для чтения) и seekp() (для записи).

– Метод tellg() возвращает текущую позицию при чтении, а tellp() — при записи.

– Это позволяет перемещаться по файлу и работать не только последовательно, но и выборочно.

7. Раскройте особенности работы с разделителями данных в текстовых файлах на C++.

– При чтении оператор >> автоматически использует пробелы, табуляции и символы новой строки как разделители.

– Для построчного чтения применяется метод getline(), который считывает строку до символа новой строки.

– При необходимости можно задавать собственные разделители и обрабатывать их вручную.

8. Дайте краткое понятие бинарного ввода-вывода и сформулируйте его отличия от текстового при организации ввода-вывода на C++.

Бинарный ввод-вывод — это работа с файлами, где данные записываются и читаются в виде последовательности байтов без преобразования в символы. **Отличия от текстового ввода-вывода:**

– В текстовом файле данные хранятся как символы, а в бинарном — как байты.

– В текстовом вводе-выводе возможны преобразования (например, числа в строки), а в бинарном данные сохраняются в исходном машинном формате.

– Бинарный ввод-вывод быстрее и эффективнее для хранения больших объёмов данных, но менее удобен для чтения человеком.